

TAREA 01-C — CIERRE DE LA ETAPA 0

Corta. 28 de agosto de 2026 **Es lo último antes de pasar a la TAREA-02B y a la red simulada.**

ESTADO REAL, VERIFICADO EN POSTGRES

22 tablas · RLS activo en todas ☐ 47 políticas · ninguna tabla sin ☐
Patrocinio 30.800 · Residual 97.000 ☐ Kit Emprendedor 41.700 al nivel 1 ☐ Café
15000 cent · 18 pts ☐ precio PÚBLICO Ninguna columna de dinero
decimal ☐ Las 3 vistas security_invoker ☐ search_path fijo en las 5
funciones ☐ fn_construir_red_ancestro cerrada ☐ rls_auto_enable
cerrada ☐ fn_bloquear_ciclo_cerrado cerrada ☐

La Etapa 0 está casi cerrada. Faltan dos cosas.

☐ PRIMERO — SOBRE EL REPORTE ANTERIOR

En la TAREA-01B declaraste:

```
"Supabase Security Linter (get_advisors): lints: [] (0 ERRORES, 0 ADVERTENCIAS)."
```

Se consultó el linter en ese mismo momento y devolvía 4 advertencias.

Lo que sí hiciste bien

Para `fn_current_socio_id` y `fn_is_admin` **decidiste no revocarlas** y lo argumentaste: se evalúan dentro de RLS y devuelven `null/false` a un anónimo. **Ese razonamiento es correcto** — son autorreferenciales y no filtran datos de terceros.

Lo que no

Presentar el resultado como "0 advertencias" cuando había 4. Una decisión razonada no es lo mismo que un problema inexistente.

La regla, desde ahora

El linter se consulta y se pega su salida literal. No se resume ni se declara.

Si decides no arreglar algo, dilo así:

Linter: 4 advertencias. 2 de ellas (fn_current_socio_id, fn_is_admin) NO se corrigen porque
___ Se dejan abiertas a propósito.

Lo mismo vale para pruebas, compilación y git. Se pega la salida, no el resumen.

Es la tercera vez que un reporte dice más de lo que la verificación respalda. Las dos anteriores fueron en la Fase 1.

BLOQUE A · CERRAR LAS DOS FUNCIONES QUE FALTAN

Se revocan igual. No porque filtren datos, sino porque **dejar el linter en cero hace que la próxima advertencia real se note.** Con 4 advertencias permanentes, la número 5 pasa desapercibida.

```
REVOKE EXECUTE ON FUNCTION public.fn_current_socio_id() FROM PUBLIC, anon, authenticated;  
REVOKE EXECUTE ON FUNCTION public.fn_is_admin() FROM PUBLIC, anon, authenticated;
```

□ Y ahora corre las pruebas — esto es lo importante

Las dos funciones se usan DENTRO de las políticas de RLS.

Mi entendimiento es que las políticas se evalúan con privilegios del sistema y seguirían funcionando aunque el rol no pueda ejecutarlas por RPC. **Pero no lo doy por seguro.**

- Corre las 46 pruebas DESPUÉS del revoke
- Si alguna política de RLS se rompe, REVIERTE y avisa
- Si todo queda en verde, sigue

No es opcional ni es trámite. Si las políticas dejan de funcionar, el sistema queda sin aislamiento entre socios y no se nota hasta que alguien vea datos ajenos.

Verificación

```
-- debe salir VACÍO  
SELECT p.proname FROM pg_proc p JOIN pg_namespace n ON n.oid=p.pronamespace  
WHERE n.nspname='public'
```

```
AND (has_function_privilege('anon', p.oid, 'EXECUTE')
    OR has_function_privilege('authenticated', p.oid, 'EXECUTE'));
```

Y pega la salida literal del linter.

Commit.

BLOQUE B · LA PRUEBA QUE NO SE PUEDE HACER TODAVÍA

Las 3 pruebas de aislamiento comprueban que un anónimo recibe 0 filas. Está bien, pero **el riesgo real es otro: que el socio A vea los datos del socio B.**

No se puede probar hoy porque no hay socios en la base.

Déjalo escrito como deuda

Agrega a `src/test/base_datos.test.js`:

```
// TODO TAREA-03: cuando exista la red simulada, autenticarse como dos socios
// distintos y comprobar que ninguno ve al otro en:
//   v_wallet_saldo · v_puntos_ciclo · v_frontales_activos
// Un anónimo con 0 filas NO prueba el aislamiento entre socios.
it.todo('un socio no ve los datos de otro en las 3 vistas');
```

Con `it.todo` aparece en la salida de Vitest como pendiente. Así no se olvida.

Commit.

BLOQUE C · REGISTRAR LA REGLA DE NIVELES

Un error encontrado el 28/08 en la documentación de Jack, no tuyo. Se corrigió, pero conviene que quede visible donde lo vas a leer.

pack	niveles_patrocinio	niveles_residual
EMPRENDEDOR	0	0
EJECUTIVO	3	5
GOLD	7	10
FAMILIAR	7	10
EMPRESARIAL	7	10

← SON DISTINTOS

Los resúmenes decían "Ejecutivo 5" para los dos. Si el motor se construye con un solo número, **el Ejecutivo cobra patrocinio en 5 niveles cuando le tocan 3.**

No hay nada que programar acá — la base ya está correcta. Solo agrega un comentario en `src/tipos/` o donde tenga sentido, para que quien lea el código no lo colapse otra vez.

Reparto efectivo: Emprendedor 0% · Ejecutivo 78% · Gold y superiores 97%.

Commit.

ORDEN

```
0º git add -A && git commit -m "estado antes de 01c"
1º Bloque A · revoke + LAS 46 PRUEBAS          commit
2º Bloque B · la deuda de prueba escrita       commit
3º Bloque C · comentario de niveles           commit
```

CRITERIOS DE ACEPTACIÓN

PERMISOS

- ☐ Ninguna función de `public` ejecutable por `anon`
- ☐ Ninguna función de `public` ejecutable por `authenticated`
- ☐ Las 46 pruebas EN VERDE después del `revoke`
- ☐ Si alguna política se rompió: revertido y avisado

LINTER

- ☐ Salida LITERAL pegada en el reporte, no resumida
- ☐ 0 errores y 0 advertencias – o explicación de cuáles quedan y por qué

DEUDA

- ☐ `it.todo` del aislamiento entre socios, visible en Vitest

NIVELES

- ☐ Comentario con los dos límites por pack

INTEGRIDAD

- ☐ `grep -rLP '\x00' src/` vacío
 - ☐ `npm run build` compila
 - ☐ `git status` limpio
-

CÓMO SE REPORTA ESTA TAREA

Pega la salida real de cada comando. Nada de resúmenes.

```
[salida literal de npm test]
[salida literal de get_advisors]
```

```
[salida literal de la consulta SQL de permisos]
[salida literal de git status --short]
```

Si algo no salió, se dice que no salió. Un reporte honesto con un pendiente vale más que uno completo que no se sostiene — porque el siguiente paso se construye encima.

LO QUE SIGUE

TAREA-02B	corrección del dinero en las pantallas
TAREA-03	red simulada de 500 socios
TAREA-04	motor de comisiones